Given

- Symbols: A, B, C
- Probabilities: $p(A) = 0.7,\ p(B) = 0.2,\ p(C) = 0.1$
- Codewords: A → `0` (length 1), B → `10` (length 2), C → `11` (length 2)

1) Average codeword length L

$$L = \sum_i p_i \ell_i = 0.7 \cdot 1 + 0.2 \cdot 2 + 0.1 \cdot 2 = 0.7 + 0.4 + 0.2 = 1.3 \text{ bits/symbol.}$$

2) Entropy H of the source (base-2)

$$H = -\sum_i p_i \log_2 p_i = -\left(0.7 \log_2 0.7 + 0.2 \log_2 0.2 + 0.1 \log_2 0.1\right).$$

Compute each term (using $\log_2$):

- $-0.7 \log_2 0.7 \approx 0.360201$
- $-0.2 \log_2 0.2 \approx 0.464386$
- $-0.1 \log_2 0.1 \approx 0.332193$

Sum:

$$H \approx 0.360201 + 0.464386 + 0.332193 = 1.15678 \text{ bits/symbol.}$$

(You can round to $H \approx 1.157$ bits/symbol.)

+ Ask anything

Problem No. 1

- Let the probabilities for the outcomes A, B and C are p(A)=0.7, p(B)=0.2, p(C)=0.1 respectively. The coding scheme for each symbol is given as follows:

Source Symbol	Codeword
A	0
B	10
C	11

Determine the average codeword length, coding efficiency, entropy of the source.

3) Coding efficiency and redundancy

Coding efficiency η is H / L:

$$\eta = \frac{H}{L} \approx \frac{1.15678}{1.3} \approx 0.8891 \approx 88.91\%.$$

Redundancy $= 1 - \eta \approx 11.09\%$.

4) Additional checks / comments

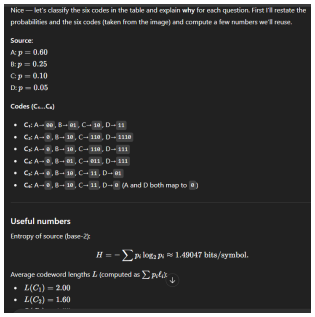
- Kraft sum: $2^{-1} + 2^{-2} + 2^{-2} = 0.5 + 0.25 + 0.25 = 1$. The code is a valid prefix code and fills the full binary tree.
- This code is the Huffman (optimal) structure for the given probabilities (lengths 1,2,2), so the average length is minimal among prefix codes. The excess over entropy is

$$L - H \approx 1.3 - 1.15678 = 0.14322 \text{ bits/symbol.}$$

Final numeric summary

- Average codeword length $L = 1.30$ bits/symbol
- Entropy $H \approx 1.157$ bits/symbol
- Coding efficiency $\eta \approx 88.9\%$ (redundancy ≈ 11.1%)

If you want, I can show the Huffman tree that produces these codewords or compute expected total bits for N symbols.



Nice — let's classify the six codes in the table and explain **why** for each question. First I'll restate the probabilities and the six codes (taken from the image) and compute a few numbers we'll reuse.

Source:
A: $p = 0.60$
B: $p = 0.25$
C: $p = 0.10$
D: $p = 0.05$

Codes (C_1…C_6)

- C_1: A→`00`, B→`01`, C→`10`, D→`11`
- C_2: A→`0`, B→`10`, C→`110`, D→`1110`
- C_3: A→`0`, B→`10`, C→`110`, D→`111`
- C_4: A→`0`, B→`01`, C→`011`, D→`111`
- C_5: A→`0`, B→`10`, C→`11`, D→`01`
- C_6: A→`0`, B→`10`, C→`11`, D→`0` (A and D both map to `0`)

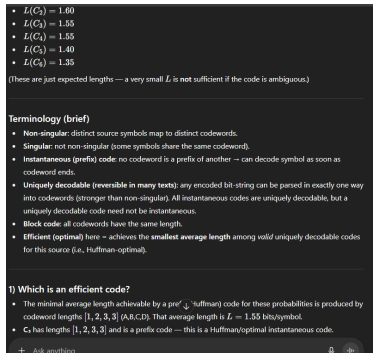
Useful numbers

Entropy of source (base-2):

$$H = -\sum p_i \log_2 p_i \approx 1.49047 \text{ bits/symbol.}$$

Average codeword lengths L (computed as $\sum p_i \ell_i$):

- $L(C_1) = 2.00$
- $L(C_2) = 1.60$



- $L(C_2) = 1.60$
- $L(C_3) = 1.55$
- $L(C_4) = 1.55$
- $L(C_5) = 1.40$
- $L(C_6) = 1.35$

(These are just expected lengths — a very small L is **not** sufficient if the code is ambiguous.)

Terminology (brief)

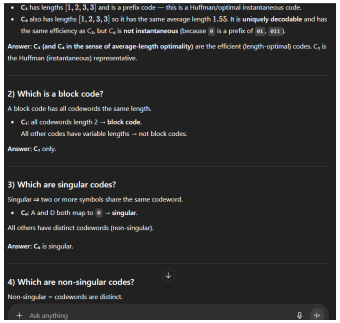
- **Non-singular**: distinct source symbols map to distinct codewords.
- **Singular**: not non-singular (some symbols share the same codeword).
- **Instantaneous (prefix) code**: no codeword is a prefix of another → can decode symbol as soon as codeword ends.
- **Uniquely decodable (reversible in many texts)**: any encoded bit-string can be parsed in exactly one way into codewords (stronger than non-singular). All instantaneous codes are uniquely decodable, but a uniquely decodable code need not be instantaneous.
- **Block code**: all codewords have the same length.
- **Efficient (optimal)** here = achieves the **smallest average length** among *valid* uniquely decodable codes for this source (i.e., Huffman-optimal).

1) Which is an efficient code?

- The minimal average length achievable by a pre' ↓ -huffman) code for these probabilities is produced by codeword lengths $[1, 2, 3, 3]$ (A,B,C,D). That average length is $L = 1.55$ bits/symbol.
- C_5 has lengths $[1, 2, 3, 3]$ and is a prefix code — this is a Huffman/optimal instantaneous code.

Ask anything

Problem No. 2



- C_5 has lengths $[1, 2, 3, 3]$ and is a prefix code — this is a Huffman/optimal instantaneous code.
- C_6 also has lengths $[1, 2, 3, 3]$ so it has the same average length 1.55. It is **uniquely decodable** and has the same efficiency as C_5, but C_6 is **not instantaneous** (because `0` is a prefix of `01`, `011`).

Answer: C_5 (and C_6 in the sense of average-length optimality) are the efficient (length-optimal) codes. C_5 is the Huffman (instantaneous) representative.

2) Which is a block code?

A block code has all codewords the same length.

- C_1: all codewords length 2 → **block code**.
  All other codes have variable lengths → not block codes.

Answer: C_1 only.

3) Which are singular codes?

Singular ⇔ two or more symbols share the same codeword.

- C_6: A and D both map to `0` → **singular**.

All others have distinct codewords (non-singular).

Answer: C_6 is singular.

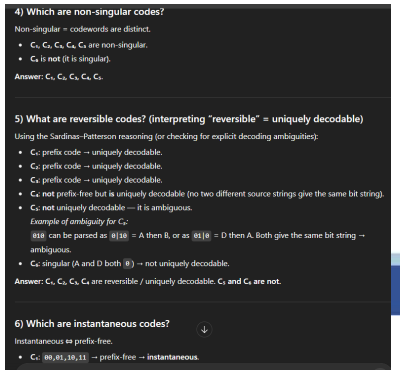
4) Which are non-singular codes?

Non-singular = codewords are distinct.

Ask anything

- Let us consider the different coding schemes given in table for encoding symbols A, B, C and D.

	Probability	C_1	C_2	C_3	C_4	C_5	C_6
A	0.6	00	0	0	0	0	0
B	0.25	01	10	10	01	10	10
C	0.1	10	110	110	011	11	11
D	0.05	11	1110	111	111	01	0



4) Which are non-singular codes?

Non-singular = codewords are distinct.

- C_1, C_2, C_3, C_4, C_5 are non-singular.
- C_6 is **not** (it is singular).

Answer: C_1, C_2, C_3, C_4, C_5.

5) What are reversible codes? (interpreting "reversible" = uniquely decodable)

Using the Sardinas–Patterson reasoning (or checking for explicit decoding ambiguities):

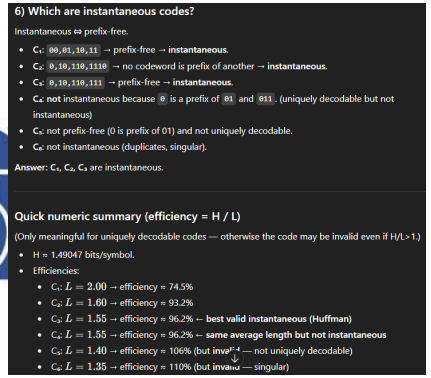
- C_1: prefix code → uniquely decodable.
- C_2: prefix code → uniquely decodable.
- C_3: prefix code → uniquely decodable.
- C_4 **not** prefix-free but **is** uniquely decodable (no two different source strings give the same bit string).
- C_6: **not** uniquely decodable — it is ambiguous.
  Example of ambiguity for C_6:
  `010` can be parsed as `0|10` = A then B, or as `0|1|0` = D then A. Both give the same bit string → ambiguous.
- C_6: singular (A and D both `0`) → not uniquely decodable.

Answer: C_1, C_2, C_3, C_4 are reversible / uniquely decodable. **C_5 and C_6 are not.**

6) Which are instantaneous codes? ↓

Instantaneous ⇔ prefix-free.

- C_1: `00,01,10,11` → prefix-free → **instantaneous**.



6) Which are instantaneous codes?

Instantaneous ⇔ prefix-free.

- C_1: `00,01,10,11` → prefix-free → **instantaneous**.
- C_2: `0,10,110,1110` → no codeword is prefix of another → **instantaneous**.
- C_3: `0,10,110,111` → prefix-free → **instantaneous**.
- C_4: **not** instantaneous because `0` is a prefix of `01` and `011`. (uniquely decodable but not instantaneous)
- C_5: not prefix-free (0 is prefix of 01) and not uniquely decodable.
- C_6: not instantaneous (duplicates, singular).

Answer: C_1, C_2, C_3 are instantaneous.

Quick numeric summary (efficiency = H / L)

(Only meaningful for uniquely decodable codes — otherwise the code may be invalid even if H/L > 1.)

- H ≈ 1.49047 bits/symbol.
- Efficiencies:
  - C_1: $L = 2.00$ → efficiency ≈ 74.5%
  - C_2: $L = 1.60$ → efficiency ≈ 93.2%
  - C_3: $L = 1.55$ → efficiency ≈ 96.2% ← **best valid instantaneous (Huffman)**
  - C_4: $L = 1.55$ → efficiency ≈ 96.2% ← **same average length but not instantaneous**
  - C_5: $L = 1.40$ → efficiency ≈ 106% (but **invalid** — not uniquely decodable)
  - C_6: $L = 1.35$ → efficiency ≈ 110% (but **invalid** — singular)

Problem No. 2 Cont…

- Determine the following:
  1. Which is an efficient code?
  2. Which is a block code?
  3. Which are singular codes?
  4. Which are non-singular codes?
  5. What are reversible codes?
  6. Which are instantaneous codes?